

Chapter 13: Text Classification & Naive Bayes

https://drive.google.com/file/d/1uB9leiG-HEAtEX9IgSXdg_DFDWvUVAT4/view

<https://drive.google.com/file/d/1sWvloE15gwzy-4MfuBntBKjSYAKbr2Fv/view>

1. Ad Hoc vs. Standing Query

› Ad Hoc Query:

This is a **one-time, immediate search**.

- You have a **temporary (transient) information need**
- You type a query into a search engine (like Google)
- You get results **right now**, and that's it

› Standing Query:

This is a **continuous or repeated search over time**.

- The query is **saved and run repeatedly**
- The system keeps **checking** for **new documents/data**
- It's **useful when information keeps updating**
- Classification using standing queries is also called **routing** or **filtering**

✚ **Key Difference:** *Ad hoc queries are one-time; standing queries are persistent and run repeatedly on growing collections.*

2. Text / Document Classification

Text classification is the process of **assigning** a **predefined set of classes/categories to new text instances**.

Input: A document d , and a fixed set of classes $C = \{c_1, c_2, \dots, c_n\}$

Output: A predicted class $c^I \in C$

› Types of Text Classification

Type	Description	Example Use Case
Manual	Hand-coded rules by human experts based on combinations of words or features.	Spam filter: blacklist OR ("dollars" AND "selected")
Supervised	A classifier is trained on labelled documents (d^I, c^I) pairs and learns to predict class for new documents.	Sentiment analysis, spam detection
Semi-supervised	Starts as supervised ; continues to learn from its own classifications over time .	News topic classification
Unsupervised	Implicitly learns features and categories without labelled training data .	Topic modelling, clustering

🔗 **Manual Classification:** Can achieve **high accuracy** when rules are carefully refined by experts, but **building and maintaining rules** is very **expensive**.

Classification Configurations

# of Outputs	Output Type	Classification Kind
1 per instance	Binary	Binary
1 per instance	Multivalued	Multi-class
n per instance	Binary	Multi-label
n per instance	Multivalued	Multi-dimensional

1 per n instances	Binary/Multivalued	Multi-instance
-------------------	--------------------	-----------------------

One-of and any-of classification

Type	Meaning
One-of	Each document gets <u>exactly one class</u> .
Any-of	A document may get <u>more than one class</u> .

Training accuracy is not enough

- The goal is high accuracy on **new test data**, not just on the training data.
- A classifier can simply memorize the training labels and look perfect on the training set, but then fail on new documents.
- We assume that training data and test data are similar or come from the same general source.

Important exam idea

A classifier that is very good on training data but poor on new data is not useful. Good text classification means generalizing to new documents.

3. Naive Bayes Classification

It is a probabilistic model based on Bayes' theorem with a "naive" **assumption** of **conditional independence between features (words)**.

$$\begin{aligned}
 \hat{y} &= \arg \max_{y \in Y} p(y | x_1, \dots, x_n) \\
 &= \arg \max_{y \in Y} \frac{p(y) p(x_1, \dots, x_n | y)}{p(x_1, \dots, x_n)} \\
 &= \arg \max_{y \in Y} p(y) p(x_1, \dots, x_n | y) \\
 &= \arg \max_{y \in Y} p(y) \prod_{i=1}^n p(x_i | y)
 \end{aligned}$$

> How likely is this class, given the words in this document? Then it chooses the class with the highest score

Core Formula

The **predicted class** is the one that **maximises the posterior probability**:

$$\hat{y} = \operatorname{argmax}_{y \in Y} P(y) \times \prod P(x^i | y)$$

Where $P(y)$ is the **prior probability** of class y , and $P(x^i | y)$ is the **conditional probability** of **each feature word** x^i **given class** y .

3.1 Multinomial Naive Bayes

- Uses **word counts / token frequencies**.
- Each document is represented as a **bag-of-words with counts**.

$$\begin{aligned}\hat{P}(c_j) &= \frac{\text{doccount}(C = c_j)}{N_{\text{doc}}} \\ \hat{P}(w_i | c_j) &= \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)} \\ \hat{P}(w_i | c) &= \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w, c) + 1)} \\ &= \frac{\text{count}(w_i, c) + 1}{\left(\sum_{w \in V} \text{count}(w, c) \right) + |V|}\end{aligned}$$

🔴 **Why Smoothing?:** Without smoothing, any word in the test set that did not appear in training for class c gives $P = 0$, making the entire product 0. **Laplace smoothing** adds 1 to all counts to **avoid zero probabilities**.

› Training Algorithm (Multinomial NB)

TRAINMULTINOMIALNB(C, D):

- Extract vocabulary V from training set D
- Count total documents N
- For each class c: count N^c documents; compute $\text{prior}[c] = N^c / N$
- Concatenate all text in class c into text c
- For each term t in V: count T^c_t (occurrences of t in class c)
- Compute $\text{condprob}[t][c] = (T^c_t + 1) / (\sum_t T^c_t + |V|)$
- Return V, prior, condprob

Testing Algorithm (Multinomial NB)

APPLYMULTINOMIALNB(C, V, prior, condprob, d):

- Extract tokens W from document d (only those in V)
- For each class c: $\text{score}[c] = \log \text{prior}[c]$
- For each token t in W: $\text{score}[c] += \log \text{condprob}[t][c]$
- Return $\text{argmax}_c \text{score}[c]$

📌 **Log Trick:** Multiplying many small probabilities causes numerical underflow. We use **log probabilities** and **sum** instead of multiply.

› Worked Example – Multinomial NB

Split	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
Training	2	Chinese Chinese Shanghai	c
Training	3	Chinese Macao	c
Training	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors: $P(c) = 3/4$, $P(j) = 1/4$

Conditional probabilities (with Laplace smoothing, $|V| = 6$):

- $P(\text{Chinese} | c) = (5+1)/(8+6) = 6/14 = 3/7$
- $P(\text{Tokyo} | c) = (0+1)/(8+6) = 1/14$
- $P(\text{Japan} | c) = (0+1)/(8+6) = 1/14$
- $P(\text{Chinese} | j) = (1+1)/(3+6) = 2/9$
- $P(\text{Tokyo} | j) = (1+1)/(3+6) = 2/9$
- $P(\text{Japan} | j) = (1+1)/(3+6) = 2/9$

[only need to calculate conditional probabilities for terms in the test doc!!]

Classification of d5 (Chinese Chinese Chinese Tokyo Japan):

- $P(c | d5) = 3/4 \times (3/7)^3 \times 1/14 \times 1/14 \approx 0.0003$
- $P(j | d5) = 1/4 \times (2/9)^3 \times 2/9 \times 2/9 \approx 0.0001$

Result: d5 is classified as China (c) since $0.0003 > 0.0001$

[SLIDES example]

$$N_c = 3, N_j = 1, N = 4$$

$$P(c) = 3/4, P(j) = 1/4 \Rightarrow \text{Prior probs.}$$

$$\bullet \text{ Vocab} = |V| = 6$$

$$P(w|c) = \frac{\text{count}(w|c) + 1}{\text{count}(c) + 1}$$

$$\bullet \text{ tot. word count in class } c = \text{count}(c) = 8$$

$$\bullet \text{ " " " " " } j = \text{count}(j) = 3$$

$\Rightarrow$ conditional probs- [only calculated for features in test doc?]

$$P(\text{chinese} | c) = \frac{5+1}{8+6} = 3/7 \quad P(\text{chinese} | j) = \frac{2}{3+6} = 2/9$$

$$P(\text{tokyo} | c) = \frac{0+1}{8+6} = 1/14 \quad P(\text{tokyo} | j) = 2/9$$

$$P(\text{japan} | c) = 1/14 \quad P(\text{japan} | j) = 2/9$$

$\Rightarrow$ choosing class:- $d5 = \text{chinese chinese chinese tokyo japan.}$

$$\bullet P(c | d5) = \cancel{3/4} \times (3/7)^3 \times 1/14 \times 1/14 \approx 0.0003 \quad \text{d5 classified as c.}$$

$$\bullet P(j | d5) = 1/4 \times (2/9)^3 \times 2/9 \times 2/9 \approx 0.0001$$

3.2 Multivariate Bernoulli Naive Bayes

Uses **binary word occurrence** (present/absent) rather than counts. Each document is a binary vector over the vocabulary.

[need to calculate conditional probabilities for all terms in V !!]

Prior probability: $\text{prior}[c] = N^c / N$

Conditional probability: $P(t | c) = (N^{c,t} + 1) / (N^c + 2)$

- $N^{c,t}$ = number of documents in class c containing term t
- N^c = total documents in class c .

> +2 appears because there are two outcomes for a term: present or absent.

> Testing Algorithm (Bernoulli NB)

APPLYBERNOULLINB(C, V, prior, condprob, d):

- Extract terms V^d from document d
- For each class c : $\text{score}[c] = \log \text{prior}[c]$
- For each term t in V :

- If $t \in V^d$: $\text{score}[c] += \log \text{condprob}[t][c]$
- Else: $\text{score}[c] += \log(1 - \text{condprob}[t][c])$
- Return $\text{argmax}_c \text{score}[c]$

🔑 **Key Difference:** Bernoulli penalises absence of a term. Multinomial ignores absent terms.

➤ Example

► **Table 13.1** Data for parameter estimation examples.

	docID	words in document	in $c = \text{China}$?
training set	1	Chinese Beijing Chinese	yes
	2	Chinese Chinese Shanghai	yes
	3	Chinese Macao	yes
	4	Tokyo Japan Chinese	no
test set	5	Chinese Chinese Chinese Tokyo Japan	?

$$\hat{P}(c) = 3/4 \text{ and } \hat{P}(\bar{c}) = 1/4$$

$$\begin{aligned} \hat{P}(\text{Chinese}|c) &= (3 + 1)/(3 + 2) = 4/5 \\ \hat{P}(\text{Japan}|c) &= \hat{P}(\text{Tokyo}|c) = (0 + 1)/(3 + 2) = 1/5 \\ \hat{P}(\text{Beijing}|c) &= \hat{P}(\text{Macao}|c) = \hat{P}(\text{Shanghai}|c) = (1 + 1)/(3 + 2) = 2/5 \\ \hat{P}(\text{Chinese}|\bar{c}) &= (1 + 1)/(1 + 2) = 2/3 \\ \hat{P}(\text{Japan}|\bar{c}) &= \hat{P}(\text{Tokyo}|\bar{c}) = (1 + 1)/(1 + 2) = 2/3 \\ \hat{P}(\text{Beijing}|\bar{c}) &= \hat{P}(\text{Macao}|\bar{c}) = \hat{P}(\text{Shanghai}|\bar{c}) = (0 + 1)/(1 + 2) = 1/3 \end{aligned}$$

The scores of the test document for the two classes are

$$\begin{aligned} \hat{P}(c|d_5) &\propto \hat{P}(c) \cdot \hat{P}(\text{Chinese}|c) \cdot \hat{P}(\text{Japan}|c) \cdot \hat{P}(\text{Tokyo}|c) \\ &\quad \cdot (1 - \hat{P}(\text{Beijing}|c)) \cdot (1 - \hat{P}(\text{Shanghai}|c)) \cdot (1 - \hat{P}(\text{Macao}|c)) \\ &= 3/4 \cdot 4/5 \cdot 1/5 \cdot 1/5 \cdot (1 - 2/5) \cdot (1 - 2/5) \cdot (1 - 2/5) \\ &\approx 0.005 \end{aligned}$$

and, analogously,

$$\begin{aligned} \hat{P}(\bar{c}|d_5) &\propto 1/4 \cdot 2/3 \cdot 2/3 \cdot 2/3 \cdot (1 - 1/3) \cdot (1 - 1/3) \cdot (1 - 1/3) \\ &\approx 0.022 \end{aligned}$$

> the classifier assigns the test document to $c' = \text{not-China}$.

[we take probabilities of terms not appearing in test doc by subtracting them w 1]

Multinomial vs. Bernoulli Comparison

Aspect	Multinomial Model	Bernoulli Model
Event model	Generation of a token	Generation of a document
Random variable	$X = t$ if t occurs at position	$U_i = 1$ if t occurs in doc
Document representation	Sequence of tokens from V	Binary vector of V
Multiple occurrences	Taken into account	Ignored
Document length	Handles longer docs better	Best for short docs
# of features	Can handle more	Best with fewer

4. Advantages of Naive Bayes

- Very **fast to train** and **classify**; **low memory** requirements
- **Robust** to irrelevant features – they cancel each other out
- **Performs** very **well when features are equally important**
- **Optimal** classifier when **independence assumption** actually holds
- A solid, reliable baseline for text classification tasks

5. Evaluation of Classification

> classification produces a matrix of its work—called **contingency matrix** or **confusion matrix**

> Confusion Matrix (Binary)

	Actual: Spam	Actual: Ham
Predicted: Spam	True Positive (TP) = 35	False Positive (FP) = 16
Predicted: Ham	False Negative (FN) = 15	True Negative (TN) = 54

> Key Metrics

Accuracy: $(TP + TN) / Total = (35 + 54) / 120 \approx 0.74$ – Fraction of correctly classified items

Error Rate: $(FP + FN) / Total = (16 + 15) / 120 \approx 0.26$ – Fraction of misclassified items

Precision: $TP / (TP + FP)$ – Fraction of predicted positives that are actually positive

Recall: $TP / (TP + FN)$ – Fraction of actual positives correctly identified

> Multi-class Classification

	C1	C2	C3	C4	C5
Pre C1	10	1	0	1	0
Pre C2	2	22	1	1	1
Pre C3	0	2	20	2	2
Pre C4	2	0	12	21	0
Pre C5	0	0	0	0	25

$$\text{Recall} = \frac{c_{ii}}{\sum_j c_{ij}}$$

↓
fraction of docs
in class i
classified correctly

$$\text{Precision} = \frac{c_{ii}}{\sum_j c_{ji}}$$

↓
fraction of docs
assigned class i
that are actually
about class i

Recall

$$C1: \frac{10}{12} \rightarrow \text{diagonal} = 0.83$$

12 → row sum

$$C2: 22/27 = 0.81$$

$$C3: 20/26 = 0.77$$

$$C4: 21/35 = 0.6$$

$$C5: 25/25 = 1$$

Precision

$$C1: \frac{10}{14} \rightarrow \text{diagonal} = 0.71$$

14 → column sum

$$C2: 22/25 = 0.88$$

$$C3: 20/33 = 0.60$$

$$C4: 21/25 = 0.84$$

$$C5: 25/28 = 0.89$$

$$\text{Accuracy: } (1 - \text{error}) = \frac{\sum_i c_{ii}}{\sum_j \sum_i c_{ij}}$$

↓
Fraction of docs
classified correctly

ACCURACY

$$= \frac{10 + 22 + 20 + 21 + 25}{136} = 98$$

sum of all diagonals

$$= 0.72$$

sum of whole table

Macroaveraging:

- Compute **precision/recall for each class independently**
- then **average**.
- **Treats all classes equally** regardless of size.

Microaveraging:

- **Aggregate** all **decisions** into a **combined contingency table**
- **then compute** metrics.
- **Dominated by performance on large/common classes.**
- TP added of all classes and filled in the TP cell of the combined table, repeat for FP, TN, FN → then we compute Precision and recall from new table!

Class 1			Class 2			Micro Ave. Table		
	Truth: yes	Truth: no		Truth: yes	Truth: no		Truth: yes	Truth: no
Classifier: yes	10	10	Classifier: yes	90	10	Classifier: yes	100	20
Classifier: no	10	970	Classifier: no	10	890	Classifier: no	20	1860

- Macroaveraged precision: $(0.5 + 0.9)/2 = 0.7$
- Microaveraged precision: $100/120 = .83$
- Microaveraged score is dominated by score on common classes

Class 1 precision = 0.5, Class 2 precision = 0.9

- Macroaveraged precision = $(0.5 + 0.9) / 2 = 0.7$
- Microaveraged precision = $100 / 120 = 0.83$

6. Supervised Learning – Datasets

Dataset Splits

- **Training set:** Used to **fit model parameters** and **select features**
- **Validation set:** Used to **tune hyperparameters**
- **Test set:** **Final evaluation** – must **never be seen during training/tuning**

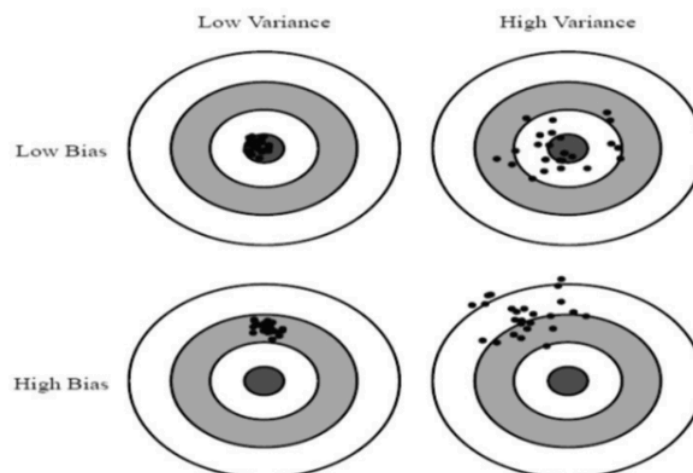
Validation Approaches

- **Holdout: Fixed split** (e.g. 70/30 train/test)
- **k-Fold Cross Validation: Divide data into k folds; train on k-1, test on 1, rotate k times, average results**

› Bias vs. Variance

- **Bias: difference between expected and actual (underfitting)**
- **Variance: Error from sensitivity to small fluctuations in training data - variability of model**

✚ **Bias-Variance Trade-off:** A model with **low bias** and **low variance** is **ideal**. **Increasing model complexity reduces bias but increases variance.** The goal is to find the right balance.



7. Feature Selection

- › Feature selection is the process of **choosing** a **subset of relevant, non-redundant features (terms)** from the full vocabulary for model training.
- › **feature** → **individual measurable property of a phenomenon being observed**
- › Choosing **informative, discriminating and independent features** is a crucial step for effective algorithms in **pattern recognition, classification** and **regression**.

Textual Features

- Words/ Lexemes/ Tokens
- Phrases/ Bi-grams/ sequences
- Graphs
- NLP based Features

› Benefits

- **Reduces dimensionality** – smaller vocabulary
- **Simplifies** the **model** and **reduces training time**
- Often **improves classification accuracy** by **removing noisy/irrelevant features**

› Feature Selection Algorithm

SELECTFEATURES(D, c, k):

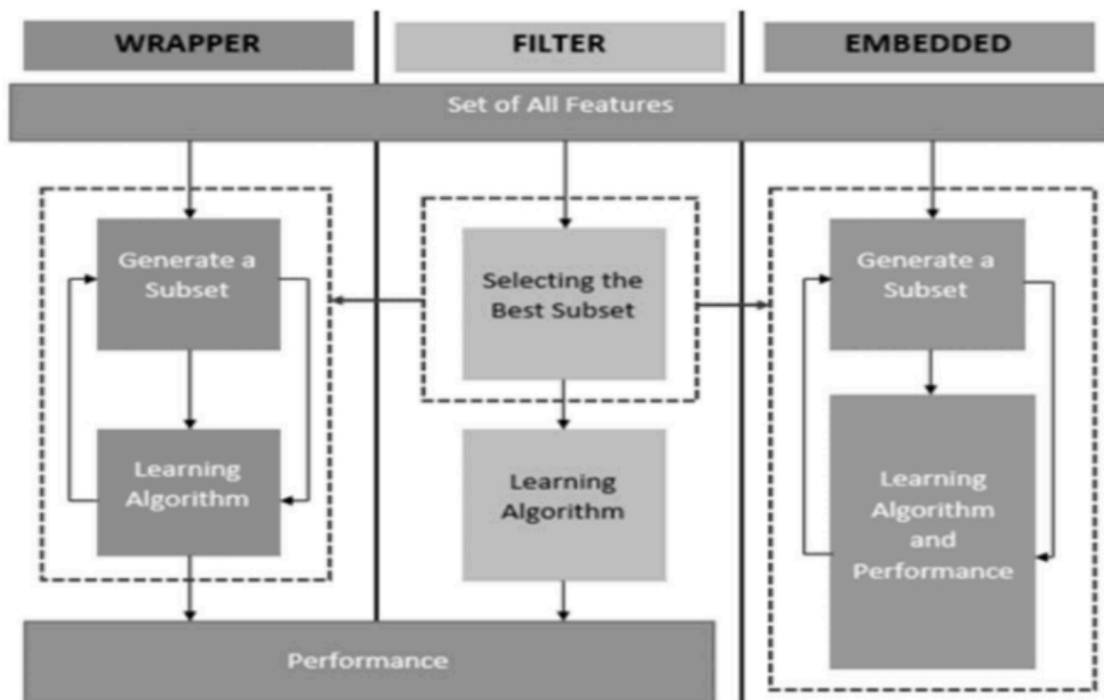
- Extract vocabulary V from D
- For each term t in V: **compute utility score** $A(t, c)$
- Return **top-k terms with highest** $A(t, c)$ values

› Three Categories of Feature Selection Methods

Method	What is it?	Speed	Overfitting	Performance
Filter	Uses a proxy statistical measure to rank features independently of the classifier	Fastest	Avoids overfitting	Sometimes misses best features
Wrapper	Uses classification accuracy of the actual learner as evaluation function – tries subsets	Slow	Prone to overfitting	Better performance

Embedded	Feature selection is built into the model building process itself	Medium	Less prone	Good performance
----------	---	--------	------------	------------------

✚ **For Text Classification: Filters** are generally preferred because Wrappers must train a classifier for each subset evaluated – **extremely expensive** with large vocabularies.



@ Wrapper:

- Use classification accuracy of some learning(weak) algorithm as their evaluation function
- Train classifier for each feature subset to be evaluated
- **Slow** → esp if number of features is high
- Not suitable for text classification

@ Filters:

- perform feature selection independently of the learning algorithm that will use the selected features.

- use an **evaluation metric** that measures the **ability** of the **feature** to **differentiate each class**.
- Less time consuming
- Widely used in text classification

> approaches to Feature Selection

7.1 Mutual Information (MI)

- measures **how much information** the **presence or absence of a term** **contributes** to **making** the **correct classification decision**

	In class c	Not in class c
Term appears	N11	N10
Term absent	N01	N00

- N11: documents containing term t and belonging to class c.
- N10: documents containing term t but not belonging to class c.
- N01: documents not containing t but belonging to c.
- N00: documents not containing t and not belonging to c.

$$I(U; C) = \sum_{e_t \in \{0,1\}} \sum_{e_c \in \{0,1\}} P(U = e_t, C = e_c) \log_2 \frac{P(U=e_t, C=e_c)}{P(U=e_t)P(C=e_c)}$$

We consider **4** cases:

Term (t)	Class (c)	Meaning
1	1	term present & class present
1	0	term present & class absent
0	1	term absent & class present
0	0	term absent & class absent

Count-Based Version

$$I(U;C) = \frac{N_{11}}{N} \log_2 \frac{NN_{11}}{N_{1.}N_{.1}} + \frac{N_{01}}{N} \log_2 \frac{NN_{01}}{N_{0.}N_{.1}} \\ + \frac{N_{10}}{N} \log_2 \frac{NN_{10}}{N_{1.}N_{.0}} + \frac{N_{00}}{N} \log_2 \frac{NN_{00}}{N_{0.}N_{.0}}$$

Instead of probabilities, we use counts:

- N_{11} : term present & class present
 - N_{10} : term present & class absent
 - N_{01} : term absent & class present
 - N_{00} : term absent & class absent
 - N : total documents
- If a word strongly indicates a class → **high MI**
 - If a word is irrelevant → **low MI**
 - Term n class are independent → **0 MI**

example:

Example 13.3: Consider the class *poultry* and the term *export* in Reuters-RCV1. The counts of the number of documents with the four possible combinations of indicator values are as follows:

	$e_c = e_{poultry} = 1$	$e_c = e_{poultry} = 0$
$e_t = e_{export} = 1$	$N_{11} = 49$	$N_{10} = 27,652$
$e_t = e_{export} = 0$	$N_{01} = 141$	$N_{00} = 774,106$

After plugging these values into Equation (13.17) we get:

$$I(U;C) = \frac{49}{801,948} \log_2 \frac{801,948 \cdot 49}{(49+27,652)(49+141)} \\ + \frac{141}{801,948} \log_2 \frac{801,948 \cdot 141}{(141+774,106)(49+141)} \\ + \frac{27,652}{801,948} \log_2 \frac{801,948 \cdot 27,652}{(49+27,652)(27,652+774,106)} \\ + \frac{774,106}{801,948} \log_2 \frac{801,948 \cdot 774,106}{(141+774,106)(27,652+774,106)} \\ \approx 0.0001105$$

7.2 Chi-Square (χ^2) Method

Chi-Square measures: **“Is the term (word) and the class dependent on each other?”**

- If they are **independent** → the word is useless
- If they are **dependent** → the word is important

> a measure of **how much expected counts E** and **observed counts N deviate from each other.**

- **High χ^2** → term and class are **dependent** (term is a useful feature)
- **Low χ^2** → term and class are **independent** (term is not useful)

> We compare:

- **Observed counts (N)** → actual data
- **Expected counts (E)** → what we **should see** if **term & class** were **independent**

👉 If **observed $\neq$ expected** → **strong relationship** → **good feature**

$$\chi^2(D, t, c) = \sum_{e_t \in \{0,1\}} \sum_{e_c \in \{0,1\}} \frac{(N_{e_t e_c} - E_{e_t e_c})^2}{E_{e_t e_c}}$$

- N = observed counts
- E = expected counts

Simplified Formula (Using counts)

$$\chi^2 = \frac{(N_{11} + N_{10} + N_{01} + N_{00})(N_{11}N_{00} - N_{10}N_{01})^2}{(N_{11} + N_{01})(N_{11} + N_{10})(N_{10} + N_{00})(N_{01} + N_{00})}$$

example:

Example 13.4: We first compute E_{11} for the data in Example 13.3:

$$\begin{aligned} E_{11} &= N \times P(t) \times P(c) = N \times \frac{N_{11} + N_{10}}{N} \times \frac{N_{11} + N_{01}}{N} \\ &= N \times \frac{49 + 141}{N} \times \frac{49 + 27652}{N} \approx 6.6 \end{aligned}$$

where N is the total number of documents as before.

We compute the other $E_{e_t e_c}$ in the same way:

	$e_{poultry} = 1$		$e_{poultry} = 0$	
$e_{\text{export}} = 1$	$N_{11} = 49$	$E_{11} \approx 6.6$	$N_{10} = 27,652$	$E_{10} \approx 27,694.4$
$e_{\text{export}} = 0$	$N_{01} = 141$	$E_{01} \approx 183.4$	$N_{00} = 774,106$	$E_{00} \approx 774,063.6$

Plugging these values into Equation (13.18), we get a X^2 value of 284:

$$X^2(\mathbb{D}, t, c) = \sum_{e_t \in \{0,1\}} \sum_{e_c \in \{0,1\}} \frac{(N_{e_t e_c} - E_{e_t e_c})^2}{E_{e_t e_c}} \approx 284$$

> step by step calculation

	Class = 1	Class = 0
Term = 1	$N_{11} = 49$	$N_{10} = 27,652$
Term = 0	$N_{01} = 141$	$N_{00} = 774,106$

$$N = 801,948$$

> observed counts

◆ Step 1: Formula

We use:

$$\chi^2 = \sum \frac{(N - E)^2}{E}$$

We compute this for all 4 cells.

◆ Step 2: Compute Expected Values

Formula:

$$E_{ij} = \frac{(\text{row total}) \times (\text{column total})}{N}$$

✓ Compute totals:

- Row totals:
 - $N_{1.} = 49 + 27,652 = 27,701$
 - $N_{0.} = 141 + 774,106 = 774,247$
- Column totals:
 - $N_{.1} = 49 + 141 = 190$
 - $N_{.0} = 27,652 + 774,106 = 801,758$

✓ Now expected counts:

1. E_{11}

$$E_{11} = \frac{27,701 \times 190}{801,948} \approx 6.6$$

2. E_{10}

$$E_{10} = \frac{27,701 \times 801,758}{801,948} \approx 27,694.4$$

3. E_{01}

$$E_{01} = \frac{774,247 \times 190}{801,948} \approx 183.4$$

4. E_{00}

$$E_{00} = \frac{774,247 \times 801,758}{801,948} \approx 774,063.6$$

◆ Step 3: Compute Each χ^2 Term

Now compute:

◆ Term 1: $(N_{11} - E_{11})^2 / E_{11}$

$$\frac{(49 - 6.6)^2}{6.6} = \frac{(42.4)^2}{6.6} = \frac{1797.76}{6.6} \approx 272.4$$

◆ Term 2: $(N_{10} - E_{10})^2 / E_{10}$

$$\frac{(27652 - 27694.4)^2}{27694.4} = \frac{(-42.4)^2}{27694.4} \approx 0.065$$

♦ Term 3: $(N_{01} - E_{01})^2 / E_{01}$

$$\frac{(141 - 183.4)^2}{183.4} = \frac{(-42.4)^2}{183.4} \approx 9.8$$

♦ Term 4: $(N_{00} - E_{00})^2 / E_{00}$

$$\frac{(774106 - 774063.6)^2}{774063.6} \approx 0.002$$

♦ Step 4: Add Everything

$$\chi^2 \approx 272.4 + 0.065 + 9.8 + 0.002 \approx 282.3 \approx 284$$

♦ Final Insight

👉 Almost ALL the χ^2 value comes from one cell: N_{11}

Why?

- Expected = 6.6
- Observed = 49 → much larger than expected

🔥 This means:

The term appears far more often with the class than randomness predicts

♦ Key Takeaway

- χ^2 becomes large when observed $\neq$ expected
- Here:
 - The word "export" strongly deviates from independence
 - So χ^2 says → important feature

7.3 Frequency-Based Feature Selection

- selects the **most common terms in each class**.
- fast and simple but selects some frequent terms that carry **no class-specific information (e.g. stopwords)**.

› Frequency can be either defined as

- **Document frequency:**
documents in class c containing term $t \rightarrow$ best for **Bernoulli** model
- **Collection frequency:**
occurrences of t in class $c \rightarrow$ best for **Multinomial** model

Features Generation

Instead of selecting from existing features, **generate higher-level (more abstract) features from raw (low-level) features**.

› Example: Build a sentence dependency graph from tokens to capture syntactic relationships.

◀ Food for Thoughts ▶

Q1. Define Text Classification and Provide a Mathematical Model

Text classification is the task of **assigning a predefined category label** from a **fixed set $C = \{c_1, c_2, \dots, c_n\}$** to an input document d .

Mathematical model: Given a document d and a set of classes C , find:

$$c^* = \underset{c \in C}{\operatorname{argmax}} P(c \mid d) = \underset{c \in C}{\operatorname{argmax}} P(c) \times P(d \mid c)$$

$$c^* = \underset{c \in C}{\operatorname{argmax}} P(c) \times \prod_l P(x^l \mid c)$$

Q2. Multinomial NB vs. Bernoulli NB

Feature	Multinomial NB	Bernoulli NB
Representation	Term frequency (count of each word)	Binary (word present/absent)
Multiple occurrences	Counted and used	Ignored
Absent terms	Not penalised	Penalised (contributes $1-P(t c)$)
Best for	Longer documents, many features	Short documents, fewer features
Smoothing formula	$(\text{count}(w,c)+1) / (\sum \text{count}(w,c)+ V)$	$(N^c_{\square}+1) / (N^c+2)$

Q3. Supervised Learning – Definition and Requirements

- machine learning paradigm where a **model is trained on labelled examples**.
- Each training example is a (document, class) pair (d, c) .

Essential Requirements

- **Labelled training data:** A set of documents with correct class labels. The quality and quantity of labels directly determines model quality.
- **Fixed set of classes:** Classes C must be predefined before training.
- **Feature representation:** Documents must be converted to a feature vector format.

- **Learning algorithm:** An algorithm that can estimate model parameters from labelled data (e.g. NB, SVM, kNN).
- **Evaluation methodology:** A way to measure model performance on unseen data (held-out test set or cross-validation).